



Booting with U-Boot

- ▶ On ARM32, U-Boot can boot `zImage` (`bootz` command)
- ▶ On ARM64 or RISC-V, it boots the `Image` file (`booti` command)
- ▶ In addition to the kernel image, U-Boot should also pass a DTB to the kernel.
- ▶ The typical boot process is therefore:
 1. Load `zImage` at address X in memory
 2. Load `<board>.dtb` at address Y in memory
 3. Start the kernel with `boot[z|i] X - Y` The - in the middle indicates no *initramfs*



Kernel command line

- ▶ In addition to the compile time configuration, the kernel behavior can be adjusted with no recompilation using the **kernel command line**
- ▶ The kernel command line is a string that defines various arguments to the kernel
 - It is very important for system configuration
 - `root=` for the root filesystem (covered later)
 - `console=` for the destination of kernel messages
 - Example: `console=ttyS0 root=/dev/mmcblk0p2 rootwait`
 - Many more exist. The most important ones are documented in [admin-guide/kernel-parameters](#) in kernel documentation.



Passing the kernel command line

- ▶ U-Boot carries the Linux kernel command line string in its `bootargs` environment variable
- ▶ Right before starting the kernel, it will store the contents of `bootargs` in the `chosen` section of the Device Tree
- ▶ The kernel will behave differently depending on its configuration:
 - If `CONFIG_CMDLINE_FROM_BOOTLOADER` is set: The kernel will use only the string from the bootloader
 - If `CONFIG_CMDLINE_FORCE` is set: The kernel will only use the string received at configuration time in `CONFIG_CMDLINE`



Passing the kernel command line

- If `CONFIG_CMDLINE_EXTEND` is set: The kernel will concatenate both strings

See the “Understanding U-Boot Falcon Mode” presentation from Michael Opdenacker, for details about how U-Boot boots Linux.



Booting from raw NAND - Results and notes

- ▶ Reference test
 - ▶ To be fair, using a zero `bootdelay` and the exact `zImage` and `dtb` size:
`setenv bootdelay 0`
`setenv bootcmd 'nand read 0x21000000 0x1a0000 0x53ac00; nand read 0x22000000 0x180000 0x6c93; bootz 0x21000000 - 0x22000000'`
 - ▶ Best result (using `grabserial`):
`[4.320618 0.000470] Please press Enter to activate this console.`
- ▶ Falcon boot test
 - ▶ Best result (using `grabserial`):
`[3.768543 0.000125] Please press Enter to activate this console.`
 - ▶ We saved 552 ms!



Passing the kernel command line

Slides: <https://bootlin.com/pub/conferences/2021/lee/> Video: <https://www.youtube.com/watch?v=LFe3x2QMhSo>



Kernel log

- ▶ The kernel keeps its messages in a circular buffer in memory
 - The size is configurable using `CONFIG_LOG_BUF_SHIFT`
- ▶ When a module is loaded, related information is available in the kernel log.
- ▶ Kernel log messages are available through the `dmesg` command (diagnostic message)
- ▶ Kernel log messages are also displayed on the console pointed by the `console=` kernel command line argument
 - Console messages can be filtered by level using the `loglevel` parameter
 - Example: `console=ttyS0 loglevel=5`
- ▶ It is possible to write to the kernel log from user space: `echo`
`"<n>Debug info" > /dev/kmsg`